

Projektna dokumentacija iz Osnova geoinformatike

Monitoring poljoprivrednih parcela

Nađa Krivokapić RA 29-2023

Andreja Vučić RA 43-2023

Jelena Šušić RA 64-2023

Opis projekta

Cilj projekta jeste pravljenje veb aplikacije za učitavanje podataka o poljoprivrednim parcelama Srema i manipulisanje istim koristeći znanja iz PostgreSQL-a i mašinskog učenja. Postupak izrade projekta se može podeliti na tri jasne celine koje čine PythonSQL, PythonGEO i PythonML.

Faza 1 - Python SQL

Kreiranje baze podataka

Koristeći šest međusobno povezanih tabela:

Tipovi zemljišta - podaci o tipu zemljišta: pH vrednost, pogodnost za useve, sadržaj organske materije

Katastarske opštine - podaci o katastarskim opštinama na području Srema: površina, broj parcele, šifra opštine

Vlasnici - fizička i pravna lica koja poseduju parcelu

Parcele - poljoprivredne parcele sa geometrijom; povezano je sa katastarskom opštinom, vlasnikom i tipom zemljišta

Usevi - usevi zasejani na parcelama sa datumima setve i žetve i statusom

Nadzor - evidencija satelitskog nadzora useva, zdravstveno stanje i geometrijske tačke

napravljena je baza podataka.

Veza između tabela realizovana koristeći strane ključeve. Tabele su međusobno joinovane:

- **parcele.id_kat_opstina** → **katastarske_opstine.id_kat_opstina** (ON DELETE CASCADE)
- **parcele.id_vlasnika** → **vlasnici.id_vlasnika** (ON DELETE SET NULL)
- **parcele.id_tip_zemljista** → **tipovi_zemljista.id_tip_zemljista** (ON DELETE SET NULL)
- **usevi.id_parcele** → **parcele.id_parcele** (ON DELETE CASCADE)
- **nadzor.id_useva** → **usevi.id_useva** (ON DELETE CASCADE)

Brisanje katastarske opštine ili useva povlači sa sobom brisanje svih zavisnih redova, dok brisanje vlasnika ili tipa zemljišta postavlja vezu na NULL vrednost. Ovim je osigurano očuvanje integriteta baze.

U okviru skripte *db_setup.py* realizovano je kreiranje same baze i tabela. Implementirane funkcije:

create_database(): proverava postojanje baze koja je definisana u DB_CONFIG. Ukoliko baza ne postoji, kreira se. Zatim se vrši konektovanje na kreiranu bazu i aktivira se PostGIS ekstenzija, što omogućava rad sa geometrijskim podacima.

drop_tables(): briše sve tabele projekta, što omogućava ponovno pokretanje inicijalizacije baze

create_tables(): izvršavanjem DDL naredbi kreira se kompletna šema baze, uključujući primarne i strane ključeve, kao i geometrijske kolone, podrazumevane vrednosti, ograničenja jedinstvenosti.

setup_database(): poziva prethodne tri

U okviru skripte *db_crud_queries.py* omogućava operacije nad podacima. Inicijalizuje se početni skup podataka od šest tabela pomoću **insert_initial_data()** funkcije.

Redosled unosa prati hijerarhiju stranih ključeva, tako što se prvo unose nezavisne tabele. Nakon unosa vrši se potvrda unetih podataka.

Implementirane su četiri osnovne operacije nad podacima:

Funkcija	Opis	SQL operacija
<code>read_all(table_name)</code>	Čita sve redove iz tabele i vraća ih kao <code>pandas.DataFrame</code>	<code>SELECT * FROM table_name;</code>
<code>insert_row(table_name, columns, values)</code>	Unosi novi red u tabelu	<code>INSERT INTO ... VALUES ...</code>
<code>update_row(table_name, set_column, new_value, where_column, where_value)</code>	Ažurira vrednost kolone za red koji zadovoljava uslov	<code>UPDATE ... SET ... WHERE ...</code>
<code>delete_row(table_name, where_column, where_value)</code>	Briše red koji zadovoljava uslov	<code>DELETE FROM ... WHERE ...</code>

Operacije koriste parametrizovane upite kako bi se zaštitili od SQL injekcije, a naziv tabela i kolona se prosleđuje direktno.

JOIN upiti se realizuju funkcijom `run_join_queries()`, koja pokreće 7 složenih upita nad bazom, kombinujući JOIN, LEFT JOIN, WHERE, ORDER BY, GROUP BY i HAVING. Upiti su sledeći:

1. **parcele_sa_zemljistem** – parcele i njihov tip zemljišta, filtrirano po opsegu pH vrednosti (BETWEEN 6.0 AND 7.5), sortirano po površini
2. **usevi_vlasnici** – trostruki JOIN useva, parcela i vlasnika, filtriran po aktivnim usevima
3. **nadzor_niski_ndvi** – nadzor useva sa NDVI vrednošću ispod 0.75, sortirano rastuće
4. **parcele_katastar** – parcele sa katastarskom opštinom (LEFT JOIN sa usevima), filtrirano po Sremskom okrugu
5. **vlasnici_agregacija** – ukupna površina parcela po vlasniku (COUNT, SUM), uz HAVING SUM(povrsina_ha) > 3.0
6. **usevi_navodnjavanje** – usevi sa navodnjavanjem, povezani sa tipom zemljišta, vlasnikom i katastarskom opštinom (četvorostruki JOIN)
7. **nadzor_tretmani** – evidencije nadzora sa primenjenim tretmanom, povezane sa usevom, parcelom i opštinom

Upiti se izvršavaju pozivom **pandas.read_sql()** i njihovi rezultati se vraćaju u obliku DataFrame objekta i ispisuju se na konzolu.

Ova faza projekta se može pokrenuti nezavisno od ostatka

```
python -m src.db_setup  
python -m src.db_crud_queries
```

Prvim pozivom kreiraju se baza i tabele, dok se drugim unose podaci, izvršava CRUD demonstracija i pokreću JOIN upiti. Alternativno, ceo deo 1 se pokreće kroz main.py ili putem dugmeta „Inicijalizuj bazu” u Flask web aplikaciji.

Faza 2 - Python GEO

U ovoj fazi izrade projekta bavimo se geoprostornom obradom podataka. Učitani su Geofabrik podaci i nad njima se primenjuju GIS overlay tehnike, kao i upiti, preuzimanjem satelitskog NDVI rastera i kreiranjem interaktivne veb mape koja objedinjuje sve navedene komponente.

Aplikacija radi sa pet vektorskih GIS slojeva, preuzetih iz Geofabrik OSM ekstrakta:

- **landuse** - poligoni korišćenja zemljišta (njive, voćnjaci, vinogradi, livade, šume, naselja i slično)
- **buildings** - poligoni objekata i zgrada
- **roads** - linije puteva
- **waterways** - linije vodotokova
- **natural** - poligoni prirodnih površina (šume, vode, vlažna zemljišta, kamenjari i slično)

Svi slojevi se filtriraju na *bounding box* Srema i transformišu u zajednički koordinatni sistem (WGS84, EPSG:4326).

U kodu je napravljena i mini demo verzija koja se pokreće za slučaj kada u projekat nisu učitani gorenavedeni .shp fajlovi. Ta demo verzija u sebi ima mnogo manje podataka, ali služi za prikaz ispravnosti rada svih prostornih upita i obrada nad slojevima.

Učitavanje podataka implementirano je u skripti *geo_loader.py* pomoću sledećih funkcija:

load_serbia_shapefiles(): vrši se provera da li *shapefile* postoji za svaki definisani sloj i učitava ga.

__enrich__layer(): dodaje kolonu sa imenom kategorije svakog vektorskog sloja

merge_shp_with_db(): spajanje podataka iz *shapefile*-ova i naše, ručno kreirane baze podataka - omogućava da se na svakoj parceli iz baze zna koji tip zemljišta je na njoj, koji putevi i vodotoci...

- Ostatak skripte implementira pozivanje NDVI rastera (vegetacijskog rastera) za površinu Srema preuzetih sa Google Earth Engine platforme. Ako program ne detektuje tražene *shapefile*-ove, poziva demo verziju rastera.

Skripta *geo_overlay.py* demonstrira različite *overlay* tehnike i prostorne upite nad podacima iz geoprostornih baza, kao što su CLIP, INTERSECTS, DIFFERENCE, BUFFER, WITHIN, OVERLAPS i DISTANCE. Rezultati ovih upita se prikazuju kroz terminal, u sekciji *GIS&Overlay* na veb aplikaciji, kao i na interaktivnoj mapi aplikacije. Na interaktivnoj mapi su svi upiti inicijalno isključeni, a korisnik ih može lako uključiti po potrebi. Svaki upit ima svoju boju na mapi (npr. Buffer - narandžasta).

Kod gorepomenute mape iskucan je u skripti *map_viewer.py*. Kroz nju je moguće sa lakoćom videti sve podatke učitane iz *shapefile*-ova i izvršiti sve moguće analize i prostorne upite nad njima. Takođe, odrađena je i vizualizacija rezultata algoritma mašinskog učenja, koji je detaljnije opisan u nastavku.

U folderu data se nalazi html kod estetskog izgleda interaktivne mape, pored importovanih *shapefile*-ova i demo verzije NDVI rastera.

Faza 3 - Python ML

Mašinsko učenje je primenjeno radi detekcije i klasifikacije tipova useva od interesa sa satelitskih snimaka. U toj skripti je odrađena konverzija podataka u vektorski format, upis u PostGIS i prikaz rezultata na mapi.

Podaci za treniranje modela su preuzeti sa Google Earth Engine platforme. Funkcija `_create_training_data_from_gee()` izvlači relevantne spektralne karakteristike sa Sentinel 2 snimaka, kao što su *ndvi* vegetacioni indeks, *red_band*, *green_band*, *blue_band* (udeo crvene, zelene, odnosno plave boje u svetlosnom spektru svakog skupa piksela), temperatura i vlažnost... Napravljena je i zaštita u slučaju kada je konektovanje na Google Cloud neuspešno, tako što se generišu demo spektralne karakteristike za treniranje klasifikacionog modela.

Oznake useva heuristički dodeljujemo na osnovu spektralnih karakteristika, pošto je *Random Forest* algoritam nadgledanog učenja, i treba nam skup labeliranih podataka za treniranje.

Klasteri, to jest mogući usevi za detekciju su: pšenica, ječa, krompir, kukuruz, soja i suncokret.

Izabrani model je *Random Forest* sa 150 stabala i maksimalnom dubinom 12, koji kao *bagging* ansambl metoda daje izuzetno dobre rezultate, jer nad nasumičnim podskupom ulaznih podataka trenira više stabala odlučivanja. Kao izlazni klaster daje klaster koji je većinska predikcija svih stabala.

Imena klastera su tekstualnog tipa, pa je odrađeno enkodovanje istih, to jest njihova prezentacija u obliku brojeva. Podaci su zatim podeljeni na train i test skup, i nakon toga skalirani. Da je skaliranje prethodilo podeli na train i test skup, došlo bi do curenja podataka. To je odrađeno kako bi svi podaci bili u istom opsegu, i kako bi i train i test skup imao istu maksimalnu vrednost. Ovime obezbeđujemo da veći podaci nemaju veći značaj samo jer su drugačije prirode.

Model je istreniran sa tačnošću 94%, što je izuzetno dobro. Izdvojeni su i najznačajniji atributi, a to su *ndvi* (*Normalized Difference Vegetation Index*) od 32.8%, *savi* (*Soil-Adjusted Vegetation Index*) od 28.6% i *ndwi* (*Normalized*

Difference Water Index) od 19.4%. Sačuvan je, kao i *scaler* i *encoder* radi mogućnosti treniranja nad drugim podacima po potrebi u folderu *models*.

Funkcija **detect_crops_on_raster** implementira istrenirani model na simuliranim satelitskim snimcima, a **_detect_crops_from_gee** to radi nad pravim satelitskim snimcima, analizirajući piksele i izračunavanjem njihovih spektralnih karakteristika.

Funkcija **save_detections_to_postgis** useve detektovane mašinskim učenjem sa satelitskih snimaka unosi u PostGIS bazu i u odgovarajući DataFrame. Preko tog DataFrame-a radi prostorne analize - izvlači statistike po tipu useva, implementira *spatial join* radi detekcije preklapanja sa *landuse* slojevima iz Geofabrik shp fajla, i funkcijom *.distance* računa udaljenost od puteva.

Implementirana je i funkcija koja omogućava izmenu atributa detektovanog objekta iz DataFrame-a ručno.

Model se automatski učitava pri pokretanju Flask aplikacije.

Faza 4 - Aplikacija

Sve funkcionalnosti su objedinjenje u jednu veb aplikaciju. Izgled aplikacije iskucan je u skripti *index.html*, koja se nalazi u folderu *templates*.

Funkcionalnost aplikacije obezbeđuje skripta *app.py*. Ovde se pozivaju sve operacije iz skripti */src* foldera i objedinjuju u Flask aplikaciju. Baza podataka i njene kolone se konvertuju u JSON format radi lakšeg prikaza na frontendu.

Link do repozitorijuma projekta (na njemu se nalazi čitav kod, sačuvani ML modeli, README fajl, ali ne i shapefile-ovi, zbog njihove veličine):

<https://github.com/krivokapicnadja/poljoprivredne-parcele.git>